Purpose

Create a set of functions to help navigate the Neutron highway in Elite Dangerous by using the Elgato Stream Deck and Stream Deck+

Let me define the behaviour I want to have a new button called "Neutron Plot" This button should work as a key on the Stream Deck, but will also be compatible with the dial (encoder).

I want the ability to store the contents of a CSV file within the current StreamDeck Elite plugin, and use Stream Deck Buttons or Dials to work through a number of different functions or display specific values.

CSV file format

The CSV file (generated from Spansh.co.uk) outputs a specific file format:

Rows:

- First Row Heading Row
- Second Row – Origin system (starting point)
- Third Row - First jump (waypoint)
- …
- Last Row – Destination system

Each row is divided into 7 columns

Columns

- Col0 = "System Name", the star system waypoint that you need to go to in the route. We need to use this.
- Col1 = "Distance", the calculated distance for the distance from the previous waypoint to the current waypoint. We need to use this.
- Col2 = "Distance Remaining", This is the calculated distance for the selected waypoint to the destination. We need to use this.
- Col3 = Ignore
- Col4 = Ignore
- Col5 = "Refuel". A flag to indicate if you should fuel at this waypoint
- Col6 = "Neutron Star". A flag to indicate whether there is a Neutron Star at this point.

Architecture Rules

Buttons and dials must not access CSV rows directly.

When importing the CSV table, the header row is removed.

Row and Column numbering starts at 0.

Buttons and dials consume only:

- NeutronPlotSnapshot
- Route API functions

All route calculations must occur in the shared route service.

The route service is the single source of truth.

ON Restart, the CSV file is re-imported. If the timestamp and file is the same as the last instance, then the WaypointCurrent value will remain unchanged. If the file timestamp or size has changed, then the WaypointCurrent value will change to 1.

Variable Name Changes from previous requirements document

A lot of variables were named differently in the original requirements document. This resulted in some confusion. It may be better to clean out all of the variables and start again. I might have a go and will need you to help me fix the things I break … like all of it :D

Phase 1 – create Persistent State

The properties of the file and the contents needs to be stored within the plugin. These properties include:

- CsvPath
- CsvLastWriteTimeUtc
- CsvFileSizeBytes //file length in bytes
- WaypointCurrent //Current waypoint number (starting from 0)

The following variables should be derived from the Waypoints

- SystemTarget = (Row {WaypointCurrent}, Col0)
- SystemPrevious = if {WaypointCurrent} = 0, “”, else (Row {WaypointCurrent} - 1, Col0)
- SystemNext = if {WaypointCurrent} = {WaypointMax}, “”, else (Row {WaypointCurrent} + 1, Col0)
- SystemDestination = (Row {WaypointMax}, Col0)
- JumpDistance = (Row {WaypointCurrent}, Col1)
- DestinationDistance = (Row {WaypointCurrent}, Col2)
- IsRefuel = (Row {WaypointCurrent}, Col5)
- IsNeutron = (Row {WaypointCurrent}, Col6)
- WaypointsMax = the total number of rows in the Waypoints table

Phase 2 – Function Creation and Variables

The following Values are calculated:

- JumpRemaining = {WaypointMax} – {WaypointCurrent}
- JumpSummary = ($”{WaypointCurrent}/{WaypointMax}”)
- StarRefuel = If {IsRefuel} = “Yes” then {RefuelStar} = “Refuel” else RefuelStar = “”
- StarNeutron = If {IsNeutron} = “Yes”, then {NeutronStar} = “Neutron” else NeutronStar = “”

- JumpPercent = {WaypointCurrent} / max(1,{WaypointMax}) * 100

The following functions are to be created:

- Route.Initialize()
  - WaypointCurrent = min(1, {WaypointMax})
  - // This sets the row to be the first system in the jump sequence
- Route.Next()
  - WaypointCurrent = Min ({WaypointCurrent} + 1, {WaypointMax})
  - // You should not be able to go further than the final Waypoint
- Route.Previous()
  - WaypointCurrent = Max ({WaypointCurrent} - 1, 0)
  - // You should only be able to go back to the Origin system
- Route.Select()
  - Copy {SystemTarget} to Clipboard
  - // Specific function to copy a value to clipboard so another operation can paste the value independently.
- Route.Clear()
  - Removes the CSV from memory,
  - set {WaypointCurrent} = 0,
  - set {WaypointMax} = 0
  - // This resets the Neutron Plot function to zero out all relevant metrics.
- CSV.Check()
  - Checks if the CSV matches the required format.
    - Check that there are more than 2 rows
    - Check that there are 7 columns
    - Possibly check that columns 1 and 2 are numbers
    - Possibly check that columns 5 and 6 are Boolean (Yes/"")
  - If fail, then Route.Clear()
- CSV.New()
  - Runs CSV.Check()
  - Checks timestamp against existing {CsvLastWriteTimeUtc} value. If same then end()
  - Checks file size against existing {CSVFileSizeBytes} value. If same then end() else
    - Set {WaypointCurrent} = 1
- CSV.Clear()
  - Clear {CsvPath}
  - Clear {CsvLastWriteTimeUtc}
  - Set {WaypointCurrent} = 0
  - Set {WaypointMax} = 0
  - Clear {Waypoints}
  - Then Route.Clear()

Phase 3 – Create Stream Deck Button

In addition, I want to be able to display values related to the route on the button display.

Phase 3.1 – Create button and CSV File connector

- Include CSV file check
    - IF CSV check fails, then display “NO ROUTE” on button
    - After 1000ms, remove the file from the property inspector
- Include the ability to clear CSV file

Phase 3.2 – Create Functions

- Add functions:
    - Initialize Route
    - Previous System
    - Next System
    - Copy Current
- Include “Blank” which means no function will trigger when button is pressed
    - Blank is default.

Phase 3.3 – Create Long-Press functions

- Add Long-Press functions
    - Copy Current
    - Clear Route
- Include “Blank” which means no function will trigger when button is long-pressed
- Long-press is when button is pressed for at least 2000ms

Phase 3.4 – Add Information displays

Enum Defined

DisplayValue
{
    Blank
    Target System Name
    Previous System Name
    Next System Name
    Distance to Target System
    Distance to Destination
    Current Jump Number
    Total Jumps
    Jumps Remaining
    Jump Summary
    Trip Percentage
    Refuel Star
    Neutron Star
}

Information Display General

- Distance to Target System and Distance to Destination should be displayed in format “##,###.0 LY”
    - Values should be shown from 0.0 LY to 99,999 LY
- Trip Percentage should be displayed in format “###.#%”
    - Values should be shown from 0.0% to 100.0%

- Labels for information displays should be displayed above the value using the following short-names:

Information Option	Display
Blank	“”
Target System Name	Target
Previous System Name	Prev
Next System Name	Next
Distance to Target System	Jump Dist
Distance to Destination	Total Dist
Current Jump Number	Jump Nbr
Total Jumps	Jumps Total
Jumps Remaining	Jumps Left
Jump Summary	Jumps
Trip Percentage	Progress
Refuel Star	Fuel Point
Neutron Star	Neutron

Button Layout

Blank Space
Info Display Upper label
Info Display Upper value
Info Display Mid label
Info Display Mid value
Info Display Lower label
Info Display Lower value
Blank space

The Label text should be regular and max 8 point font
The value text should be bold and max 20 point font but auto-shrink to fit

Information Display Upper

- Add Information display for “Info Upper”
- Add Information text Colour
    - Default color is #FFFFFF
- Add a selector labelled “Info Upper”
    - Use the Enum “DisplayValue”
- Include “Blank” which displays nothing
    - Blank is default

Information Display Mid

- Add Information display for “Info Mid”
- Add Information text Colour
    - Default color is #FFFFFF
- Add a selector labelled “Info Mid”
    - Use the Enum “DisplayValue”
- Include “Blank” which displays nothing
    - Blank is default

Information Display Lower

- Add Information display for “Info Lower”
- Add Information text Colour
    - Default color is #FFFFFF
- Add a selector labelled “Info Lower”
    - Use the Enum “DisplayValue”
- Include “Blank” which displays nothing
    - Blank is default

Button Properties

- Title (Default {function name})
- CSV File [Choose File] (Default no file)
    - Clear File [button]
- Function
    - Blank (default)
    - Initialize Route
    - Previous System
    - Next System
    - Copy Current
- Function Long press
    - Blank (default)
    - Copy Current
    - Clear Route
- Information Upper
    - Color //Color selector
    - Info:
        - {DisplayValue}
- Information Mid
    - Color //Color selector
    - Info:
        - {DisplayValue}
- Information Lower
    - Color //Color selector
    - Info:
        - {DisplayValue}

Button/Key Matrix

Name	Type	Action
CSV File	File Chooser	On entry runs CSV.New()
Clear File	Button	On Press runs CSV.Clear
Initialize Route	Function	Route.Initialize()
Previous System	Function	Route.Previous()

Next System	Function	Route.Next()
Copy Current	Function	Route.Select()
Blank (default)	LONG Press Function	Null
Copy Current	LONG Press Function	Route.Select()
Clear Route	LONG Press Function	Route.Clear()
Blank (default)	Information	Null
Target System Name	Information	{SystemTarget}
Previous System Name	Information	{SystemPrevious}
Next System Name	Information	{SystemNext}
Distance to Target System	Information	{JumpDistance}
Destination Distance	Information	{DestinationDistance}
Current Jump Number	Information	{WaypointCurrent}
Total Jumps	Information	{WaypointMax}
Jumps Remaining	Information	{JumpRemaining}
Jump Summary	Information	{JumpSummary}
Trip Percentage	Information	{JumpPercent}
Refuel at Target	Information	{StarRefuel}
Neutron at Target	Information	{StarNeutron}

Phase 4 – Encoder/Dial Function

Support the Dial functionality of the Stream Deck +

Dial Properties

- Title (Default {function name})
- Anti-Clockwise Function
  - Previous System
- Clockwise Function
  - Next System
- Press Function
  - Copy Current
- Function Long press
  - Blank (default)
  - Initialize Route
  - Clear Route
- Touch Display Detail
  - Text Color //Color selector
  - Info:
    - Blank (Default)
    - Target System Name
    - Previous System Name
    - Next System Name
    - Distance To Target System
    - Distance to Destination
    - Jump Summary
    - Trip Percentage

- Refuel at Target
- Neutron at Target

Dial Matrix

Name	Type	Action
Initialize Route	Function	Route.Initialize()
Previous System	Function	Route.Previous()
Next System	Function	Route.Next()
Copy Current	Function	Route.Select()
Blank (default)	LONG Press Function	Null
Initialize Route	LONG Press Function	Route.Initialize()
Clear Route	LONG Press Function	Route.Clear()
Blank (default)	Information	Null
Target System Name	Information	{SystemTarget}
Previous System Name	Information	{SystemPrevious}
Next System Name	Information	{SystemNext}
Distance to Target System	Information	{JumpDistance}
Distance to Destination	Information	{DestinationDistance}
Jump Summary	Information	{JumpSummary}
Trip Percentage	Information	{JumpPercent}
Refuel at Target	Information	{StarRefuel}
Neutron at Target	Information	{StarNeutron}

Future Phases

- Replace CSV Distance with live game calculation
- Read current system from Elite journal/status files
- Read target system from Navigation.json
- Calculate actual remaining route distance

Potential Additional Enhancements

- Auto-advance route after successful FSDJump
- Route progress percentage and possible display on touch strip
- ETA calculations (based on current Jumps taken and remaining jumps)
- Time calculations (Total Trip time, Average Jump time)
- Trip Meter (Total distance travelled, Average Jump Length, True distance travelled – as you might not be jumping directly towards your destination)
- Fuel remaining integration
- Spansh route import directly from URL
- Separate Route Status Button Summary – Only displays Navigation details
    - Current Target
    - Jump Summary
    - Refuel Indicator
    - Neutron Indicator